

Week 08

Extras

Cảm ơn thầy Trần Duy Quang đã cung cấp template cho môn học
Cảm ơn thầy Hồ Tuấn Thanh và thầy Đỗ Nguyên Kha đã xây dựng nội dung thực hành



Lecturer: **TS. Lê Thị Nhân.**
Lab TA: **ThS. Nguyễn Hải Đăng.**

1

Notes

Put **ALL** SQL statements to **ONE** text (SQL) document names student ID 2012xxx_yy.sql (yy is a self-grading mark 01-10):

- Comment Student ID and Full name on top of file
- Run ALL lines in file without any error
- USE the CORRECT database

2 Lab

Database name: Company of Week 4 (Ref: Fundamentals of database systems)

With additional data to make employee #2 works on all projects

```
INSERT INTO EmployeeProjects
VALUES(2, 1, 6.5)
INSERT INTO EmployeeProjects
VALUES(2, 30, 5.5)
```

Aggregate Functions (COUNT, SUM, AVG...)

```
SELECT EmployeeID, COUNT(ProjectID)
FROM EmployeeProjects
GROUP BY EmployeeID
HAVING COUNT(ProjectID) > 1
```

All columns except aggregate function in **SELECT** must be in **GROUP BY**. Condition of the aggregate function should be declared in the **HAVING** clause.

LIMIT and ORDER BY

```
SELECT TOP 1 ID, FirstName, LastName, Salary
FROM Employees
ORDER BY Salary DESC
```

Relational Division

Query: Find all employees who work (1) on all projects (2)?

Determine parts of the division:

- **Dividend** (1): (All) Employees who work => **EmployeeProjects** table
- **Divisor** (2): (All) Projects => **Projects** table

Using NOT EXISTS

```
SELECT DISTINCT x.EmployeeID
FROM EmployeeProjects AS x
WHERE NOT EXISTS (
```

```
SELECT *
FROM Projects AS y
WHERE NOT EXISTS (
    SELECT *
    FROM EmployeeProjects AS z
    WHERE z.EmployeeID = x.EmployeeID
        AND z.ProjectID = y.ID
)
```

x and **z** are the dividend table. **y** is the divisor.

Using EXCEPT

```
SELECT EmployeeID
FROM EmployeeProjects
EXCEPT SELECT EmployeeID FROM (
    SELECT EmployeeID, ID AS ProjectID
    FROM (
        SELECT EmployeeID FROM EmployeeProjects
    ) AS x, Projects
    EXCEPT
    SELECT EmployeeID, ProjectID FROM EmployeeProjects
) as y
```

Note: Using temporary tables is not recommended.

Using Aggregate Functions

```
SELECT EmployeeID
FROM EmployeeProjects
-- WHERE ProjectID IN (SELECT ID FROM Projects)
GROUP BY EmployeeID
HAVING COUNT(ProjectID) = (SELECT COUNT(ID) FROM Projects)
```

The commented **WHERE** clause must be declared when the divisor is a **strict** subset of **Projects**.

DELETE/UPDATE with conditions on tables

Query: Delete all employees who have no Supervisor on projects? => Employee #8

Using EXISTS

```
DELETE
FROM EmployeeProjects
WHERE EXISTS (
    SELECT *
    FROM Employees
    WHERE EmployeeID = ID
        AND SupervisorID IS NULL
)
```

Using IN

```
DELETE
FROM EmployeeProjects
WHERE EmployeeID IN (
    SELECT ID
    FROM Employees
    WHERE SupervisorID IS NULL
)
```

Using WITH and IN

```
WITH NoSupervisor(ID) AS (
    SELECT ID
    FROM Employees
    WHERE SupervisorID IS NULL
)
DELETE
FROM EmployeeProjects
WHERE EmployeeID IN (SELECT ID FROM NoSupervisor)
```

SELECT with locks

Database name: Bank of Week 7

Select with an update lock (**UPDLOCK**) will lock the rows which matches criteria until the transaction completed. Other transactions with or without a lock may read those rows or insert new ones that match criteria but do not delete or update.

Select with a hold lock will lock other transactions not to insert, update or delete.

```
SELECT * FROM table_name WITH (UPDLOCK) WHERE ID = 1  
SELECT * FROM table_name WITH (HOLDLOCK) WHERE ID = 1
```

Use the READ COMMITTED isolation level and select with locks to re-implement Week 7 homework.